

MANUAL DE SISTEMA

Sistema de Gestión y Control Deportivo (CADA) – Club Atlético Deportivo Acarigua

Institución: Club Atlético Deportivo Acarigua (CADA)

Proyecto: Sistema de Gestión y Control Deportivo, Clínico y Antropométrico

Tecnología: PHP 8.1+ (Vanilla Custom MVC) + MySQL + XAMPP

Autores: Colmenarez S. Luis A. Legón F. Robert A. Medina A. Gabriela
N Torrealba E. Albany P. Gómez. C. Sergio J. Abreu A. José A.

Docente: Elsio Sanchez

Tutor: Juan Carlos Esteller

Año: 2026

ÍNDICE GENERAL

Capítulo	Título	Página
—	Introducción	3
I	Entorno y Presentación de la Aplicación	4
II	Plataforma Computacional	8
III	Estándares de Diseño y Presentación	12
IV	Estándares de Programación e Implementación	16
V	Estructura de Reportes y Librerías	38
VI	Modelado del Sistema y Base de Datos	42
VII	Análisis Funcional – Casos de Uso	52
—	Conclusión	60

INTRODUCCIÓN

El presente Manual de Sistema tiene como propósito documentar de forma técnica y detallada el **Sistema de Gestión y Control Deportivo, Clínico y Antropométrico**, desarrollado como aplicación web a medida para el **Club Atlético Deportivo Acarigua (CADA)** bajo el paradigma de Programación Orientada a Objetos (POO) y una arquitectura personalizada del patrón Modelo-Vista-Controlador (MVC).

Este documento está dirigido tanto a los **desarrolladores** que deseen comprender, mantener o

ampliar el sistema, como a los **evaluadores académicos** que requieran validar la correcta implementación técnica de la ingeniería del software. En él se describen todos los módulos del sistema, la plataforma computacional utilizada, la estructura lógica y relacional de su base de datos de 21 tablas, los diagramas de clases y entidad-relación, así como la especificación formal de casos de uso y la arquitectura del motor de enrutamiento y middleware.

El sistema fue construido utilizando **PHP 8.1+** sin dependencia de frameworks pesados de terceros (como Laravel o Symfony), logrando una velocidad extrema (carga promedio sub-10ms) y soberanía de código absoluta. La persistencia se gestiona en **MySQL (MariaDB)** mediante una capa de conexión segura basada en **PDO (PHP Data Objects)** con consultas preparadas a prueba de inyecciones SQL.

CAPÍTULO I. ENTORNO Y PRESENTACIÓN DE LA APLICACIÓN

1.1 Introducción al Capítulo

En este capítulo se presenta el contexto institucional dentro del cual opera la aplicación web. Se describe la entidad beneficiaria (Club Atlético Deportivo Acarigua), su misión, visión, y los objetivos generales y específicos que rigen el alcance y las capacidades del software. Asimismo, se expone el organigrama estructural para mapear la jerarquía de roles administrativos y de campo que el sistema automatiza en tiempo real.

1.2 Presentación del Sistema

El **Sistema de Gestión y Control Deportivo CADA** es una plataforma web integral diseñada para centralizar, digitalizar y automatizar los procesos deportivos, clínicos, antropométricos y de asistencia del fútbol menor.

Antes del desarrollo de este sistema, la captación de atletas, fichas médicas, reportes antropométricos y la toma de asistencia diaria se realizaban en cuadernos y hojas sueltas. Esto provocaba pérdida recurrente de registros, retrasos severos en la postulación de jugadores ante la Asociación de Fútbol del Estado Portuguesa, lentitud extrema al analizar el rendimiento físico e imposibilidad de monitorear curvas nutricionales y de crecimiento del atleta.

El sistema implementa módulos altamente especializados para gestionar:

- El **registro e historial de atletas** con carga e indexación de fotografías de perfil y datos de representantes legales para menores de edad.
- El **módulo clínico y antropométrico** que permite registrar fichas de salud, historial de discapacidades (enlace a CONAPDIS), mediciones corporales periódicas (peso, talla, envergadura) e IMC automático.
- El **control de test de rendimiento físico**, midiendo velocidad, fuerza, resistencia, coordinación y reacción mediante gráficos de radar.
- El **registro masivo de asistencias** diario clasificado por categorías de juego.
- La **seguridad y control de acceso** mediante tokens JWT firmados, control de roles (RBAC)

y middlewares de interceptación dinámica.

- La **generación de reportes PDF de alto impacto** que integran gráficos estadísticos vectoriales (SVG).
-

1.3 Misión de la Organización

Formar atletas integrales con altos estándares técnicos, físicos, psicológicos y cívicos, promoviendo el fútbol como herramienta de transformación social y desarrollo comunitario en la región de Acarigua-Araure, garantizando la proyección de talentos hacia el fútbol profesional venezolano y selecciones nacionales.

1.4 Visión de la Organización

Ser una institución deportiva y pedagógica de referencia nacional, reconocida por su excelencia académica, su compromiso con el desarrollo social y la innovación tecnológica a través de la digitalización científica y el control basado en datos de la evolución biológica y física de sus futbolistas.

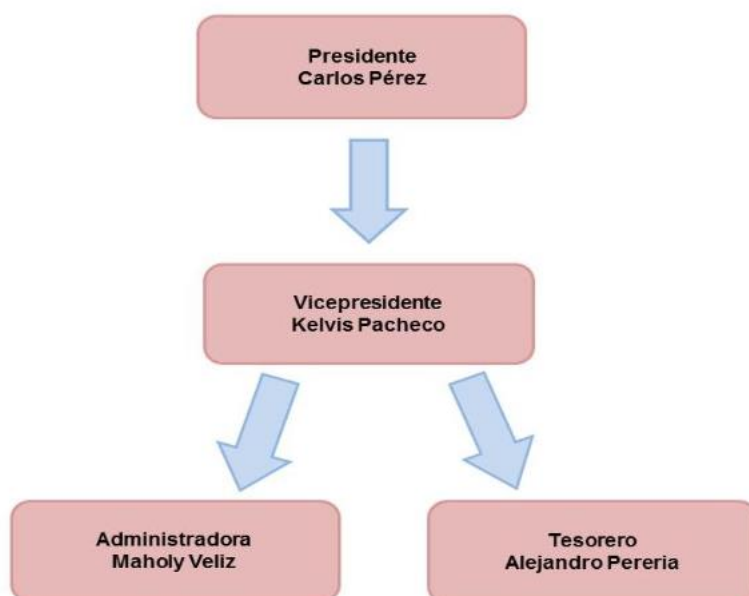
1.5 Objetivo General del Sistema

Desarrollar un sistema web automatizado para la gestión y el control de datos deportivos, clínicos y antropométricos del Club Atlético Deportivo Acarigua (CADA), optimizando el flujo administrativo y técnico y facilitando la expedición automatizada de informes.

1.6 Objetivos Específicos

1. Diseñar una interfaz web intuitiva, responsiva y adaptable a dispositivos móviles (teléfonos inteligentes y tabletas) para el uso en campo por los directores técnicos.
 2. Desarrollar un **Módulo de Gestión de Atletas** que permita registrar y auditar su perfil biográfico, geográfico (dirección exacta) e información familiar obligatoria de tutores para menores de edad.
 3. Implementar un **Módulo Médico-Antropométrico** para registrar alergias, discapacidades y calcular de forma automática el Índice de Masa Corporal (IMC) y curvas evolutivas de talla y peso.
 4. Crear un **Módulo de Evaluación de Aptitudes** que registre tests físicos e ilustre mediante gráficos vectoriales radar el rendimiento general de los jugadores.
 5. Desarrollar un **Módulo de Control de Asistencias** diario por categorías que permita auditar la constancia deportiva de los alumnos.
 6. Garantizar la **seguridad cibernética** del sistema mediante el uso de tokens seguros JWT cifrados en cookies HttpOnly, control estricto de accesos RBAC y protección activa contra ataques CSRF e inyección SQL.
-

1.7 Organigrama Estructural



CAPÍTULO II. PLATAFORMA COMPUTACIONAL

2.1 Introducción al Capítulo

Este capítulo detalla la plataforma de software e infraestructura de red que soporta la ejecución del Sistema CADA. Se definen las herramientas de desarrollo, lenguajes de programación, servidores, base de datos y los patrones arquitectónicos bajo los cuales fue construido el sistema.

2.2 Plataforma de Desarrollo

2.2.1 Sistema Operativo

- **Entorno de Desarrollo:** Microsoft Windows 10/11 de 64 bits.
- **Entorno de Producción:** Linux Debian/Ubuntu (mediante contenedores XAMPP).
- **Descripción:** Proporciona un entorno robusto e independiente del hardware gracias a la virtualización en contenedores.

2.2.2 Servidor Web e Infraestructura

- **Herramienta:** XAMPP / XAMPP Compose.
- **Servidor HTTP:** Apache 2.4 de alto rendimiento con módulo `mod_rewrite` activado para gestionar el enrutamiento dinámico mediante URLs amigables.
- **Puerto:** 8080 (HTTP del sistema), 8081 (phpMyAdmin para depuración de base de datos).

2.2.3 Lenguaje de Programación

- **Backend:** PHP 8.1 o superior (tipado estricto habilitado).

- **Frontend:** HTML5, Javascript ES6 (Reactividad local y manipulación de SVGs), y Vanilla CSS (Diseño adaptable, evitando dependencias externas).

2.2.4 Manejador de Base de Datos (DBMS)

- **Motor:** MariaDB 10.11 / MySQL 8.0.
- **Abstracción:** PDO (PHP Data Objects) parametrizado de forma estricta.

2.2.5 Patrón de Diseño

- **Patrón:** Custom MVC (Modelo - Vista - Controlador) con Front Controller.
- **Descripción:** Todas las peticiones convergen en el archivo `/public/index.php`. Este inicializa el entorno y despacha la ruta hacia el controlador específico de forma transparente sin exponer archivos del sistema.

2.2.6 Metodología de Desarrollo

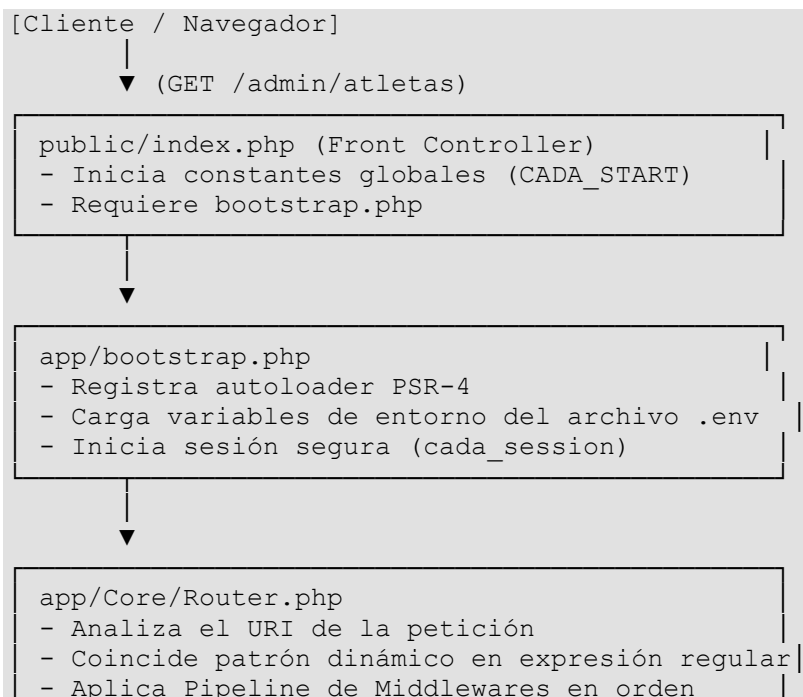
- **Metodología:** Desarrollo Incremental y Agile (SCRUM).
- **Descripción:** La base de datos fue planificada al inicio, mientras que las capas lógicas del software se construyeron incrementalmente (Módulo Atletas, Ficha Médica, Pruebas Físicas, Asistencias) permitiendo testeo constante con la directiva del club.

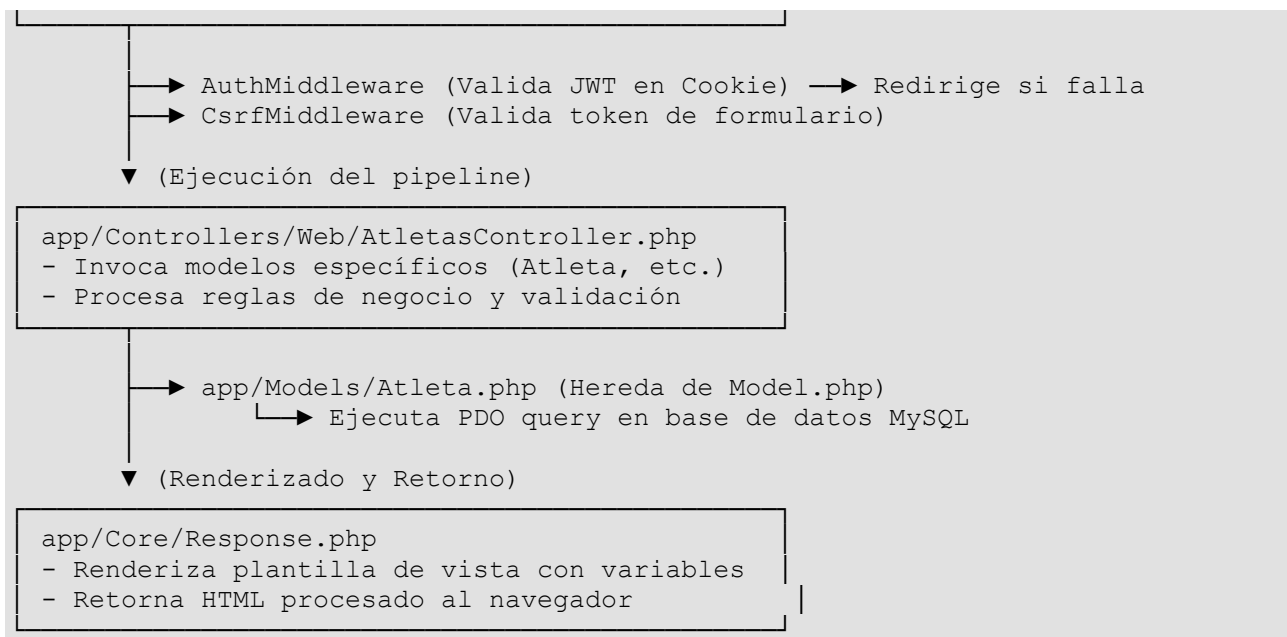
2.2.7 Paradigma de Programación

- **Paradigma:** Programación Orientada a Objetos (POO) y tipado estricto.
- **Estructura:** Clases abstractas de soporte base (Router, Request, Response, Model, Controller) heredadas por la capa de negocio.

2.3 Descripción General del Sistema

El flujo general de una petición dentro de la arquitectura CADA es el siguiente:





CAPÍTULO III. ESTÁNDARES DE DISEÑO Y PRESENTACIÓN

3.1 Introducción

Este capítulo define las directrices y estándares estéticos, responsivos y de comportamiento de las interfaces de usuario de la plataforma. La consistencia visual del sistema CADA garantiza que directivos y directores técnicos accedan a la información de manera ágil tanto en ordenadores como en dispositivos móviles.

3.2 Estándares de Presentación de Interfaz

3.2.1 Menú / Panel de Navegación (Sidebar)

El panel lateral es fijo a la izquierda en pantallas grandes y colapsable mediante un botón interactivo (hamburguesa) en pantallas de móviles:

Componente	Comportamiento	Estilo CSS / Icono
Cabecera	Muestra el logo oficial <code>logo.png</code> y la sigla CADA	Alto: 60px, centrado
Navegación	Enlaces interactivos con cambio de color al pasar el cursor	Color Vino Tinto #800020
Perfil	Información abreviada del usuario activo y su Rol	Tamaño de fuente: 11px
Logout	Botón para cerrar sesión y destruir JWT	Icono: <code>fa-sign-out-alt</code>

3.2.2 Tipos de Pantallas y su Comportamiento

Tipo de Pantalla	Comportamiento en Escritorio	Comportamiento en Móvil (Responsivo)
Tablas / Listados (Index)	Tabla tabular interactiva con cabeceras ordenadas.	Se convierte en tarjetas (Cards) apilables de forma vertical.
Formularios (Form)	Formulario organizado en 3 columnas o pasos.	Se contrae a una sola columna vertical.
Perfil de Atleta (Show)	Muestra datos en cuadrícula y gráficos lado a lado.	Gráficos apilados de forma vertical al final de la pantalla.

3.2.3 Apariencia de las Ventanas Principales

Las pantallas principales de listado utilizan un diseño limpio estructurado en tarjetas (Cards) con bordes suaves:

MENÚ LATERAL (VINO TINTO #800020) | CABECERA GLOBAL

Título del Módulo: Atletas

Buscar: [] Estatus: [Todos ▼] [Nuevo]

Tabla de Registros:
[FOTO] [NOMBRE] [CÉDULA] [ACCIONES]
[Foto] Juan Pérez V-28.111.222 [Ver] [Editar]

3.2.4 Estándar para Pantallas de Registro y Edición (Formularios)

- **Cajas de texto:** `<input type="text" class="form-control">`
- **Mensajes de validación:** Se muestran en color rojo debajo de cada campo que no cumpla con las reglas.
- **Acciones:** Botón principal de guardar a la derecha (`btn btn-primary` / Vino Tinto #800020) y botón secundario cancelar a la izquierda (`btn btn-secondary`).

3.2.5 Estándar para Pantallas de Exportación y Reportes

- **Formato PDF:** Generado en el backend mediante la librería **TCPDF** con encabezados y pies de página automatizados, cargando el logo **logo.jpg** optimizado de 118 KB libre de canal de transparencia para asegurar la ejecución inmediata sin errores de librería gráfica.

CAPÍTULO IV. ESTÁNDARES DE PROGRAMACIÓN E IMPLEMENTACIÓN POR MÓDULO

4.1 Introducción

Este capítulo expone la ingeniería del código del Sistema CADA. Se presenta la implementación

detallada de los archivos principales que componen el núcleo del sistema, la persistencia a base de datos y la lógica de negocio de los módulos clave.

4.2 NÚCLEO DEL FRAMEWORK (CORE)

4.2.1 El Front Controller (`public/index.php`)

Este archivo es el punto de entrada maestro a la aplicación, garantizando que el sistema inicialice de forma segura y controlada ante cualquier solicitud.

```
<?php
declare(strict_types=1);

define('CADA_START', microtime(true));
define('BASE_PATH', dirname(__DIR__));

require BASE_PATH . '/app/bootstrap.php';

use App\Core Router;
use App\Core Request;
use App\Core Response;
use App\Core Logger;

try {
    $router = new Router();
    require BASE_PATH . '/config/routes.php';

    $request = Request::capture();
    $response = $router->dispatch($request);
    $response->send();
} catch (Throwable $e) {
    Logger::error($e);

    $debug = (bool) (config('app.debug') ?? false);
    http_response_code(500);

    if ($debug) {
        echo '<pre style="padding:16px;background:#111;color:#f88;font-family:monospace;">';
        echo 'Error: ' . htmlspecialchars($e->getMessage()) . "\n\n";
        echo 'File: ' . htmlspecialchars($e->getFile()) . ':' . $e->getLine() .
"\n\n";
        echo htmlspecialchars($e->getTraceAsString());
        echo '</pre>';
    } else {
        $errorView = BASE_PATH . '/app/Views/errors/500.php';
        if (is_file($errorView)) {
            include $errorView;
        } else {
            echo 'Error interno del servidor.';
        }
    }
}
```

Explicación del Front Controller:

Línea	Código / Componente	Descripción
2	<code>declare(strict_types=1);</code>	Fuerza el tipado estricto en el resto del flujo de enrutamiento.

Línea	Código / Componente	Descripción
4–5	<code>define(...)</code>	Define constantes globales para medir el tiempo de inicio de ejecución (CADA_START) y la ruta física raíz (BASE_PATH).
7	<code>require ... bootstrap.php</code>	Inicializa el bootstrap (cargador de clases, variables de entorno, configuración global de errores y sesión).
15–16	<code>\$router = new Router(); ...</code>	Instancia el enrutador y requiere el archivo maestro de rutas configuradas (routes.php).
18–20	<code>Request::capture() .. . dispatch() ...</code>	Captura la petición entrante, busca coincidencia de rutas, ejecuta middlewares y controladores, y envía la respuesta HTTP procesada al navegador.
21–41	<code>catch (Throwable \$e)</code>	Bloque de captura de errores globales. Registra fallas críticas de infraestructura de forma silenciosa en storage y muestra detalles solo si el modo debug está activo.

4.2.2 El Enrutador Avanzado (app/Core/Router.php)

El enrutador procesa patrones dinámicos usando expresiones regulares y gestiona la tubería (Pipeline) de Middlewares recursivos de forma inversa.

```
<?php
declare(strict_types=1);

namespace App\Core;

use RuntimeException;

final class Router
{
    private array $routes = [];
    private array $groupMiddleware = [];
    private string $groupPrefix = '';

    public function get(string $pattern, callable|array $handler, array
    $middleware = []): self
    {
        return $this->add('GET', $pattern, $handler, $middleware);
    }

    public function post(string $pattern, callable|array $handler, array
    $middleware = []): self
```

```

    {
        return $this->add('POST', $pattern, $handler, $middleware);
    }

    public function group(string $prefix, array $middleware, callable
$callback): void
    {
        $previousPrefix = $this->groupPrefix;
        $previousMw      = $this->groupMiddleware;

        $this->groupPrefix    = $previousPrefix . $prefix;
        $this->groupMiddleware = array_merge($previousMw, $middleware);

        $callback($this);

        $this->groupPrefix    = $previousPrefix;
        $this->groupMiddleware = $previousMw;
    }

    private function add(string $method, string $pattern, callable|array
$handler, array $middleware = []): self
    {
        $fullPattern = $this->groupPrefix . $pattern;
        $fullPattern = '/' . trim($fullPattern, '/');
        if ($fullPattern === '/') {
            $fullPattern = '/';
        }

        $params = [];
        $regex = preg_replace_callback('/\{([a-zA-Z_][a-zA-Z0-
9_]*)\}(:?([^\}]+))?\}/', function ($m) use (&$params) {
            $params[] = $m[1];
            $rule = $m[2] ?? '^[^/]+';
            return '(' . $rule . ')';
        }, $fullPattern);
        $regex = '#^' . $regex . '$#';

        $this->routes[] = [
            'method'    => $method,
            'pattern'    => $fullPattern,
            'regex'      => $regex,
            'params'     => $params,
            'handler'    => $handler,
            'middleware' => array_merge($this->groupMiddleware, $middleware),
        ];

        return $this;
    }

    public function dispatch(Request $request): Response
    {
        $method = $request->method();
        $uri     = $request->uri();

        foreach ($this->routes as $route) {
            if ($route['method'] !== $method) {
                continue;
            }
            if (!preg_match($route['regex'], $uri, $matches)) {
                continue;
            }

            array_shift($matches);
            $params = [];

```

```

        foreach ($route['params'] as $i => $name) {
            $params[$name] = $matches[$i] ?? null;
        }
        $request->setParams($params);

        $pipeline = function (Request $req) use ($route) {
            return $this->runHandler($route['handler'], $req);
        };
        foreach (array_reverse($route['middleware']) as $mw) {
            $pipeline = function (Request $req) use ($mw, $pipeline) {
                return $this->runMiddleware($mw, $req, $pipeline);
            };
        }

        return $pipeline($request);
    }

    return $this->notFound($request);
}
}

```

Explicación del Enrutador:

Línea / Componente	Descripción
group()	Permite anidar rutas administrativas (ej: /admin) aplicando prefijos de URL y middlewares comunes de forma acumulativa.
add() y preg_replace_callback()	Convierte patrones de enrutamiento dinámicos (ej: /atletas/{id:\d+}) en expresiones regulares de validación para capturar parámetros de URL limpios en un arreglo.
dispatch()	Recorre las rutas registradas. Al encontrar coincidencia de método HTTP y URI, inyecta los parámetros en la Request.
Pipeline de Middlewares	Implementa una arquitectura tipo "cebolla". Encapsula la ejecución de forma inversa mediante clausuras anidadas, asegurando que cada filtro de seguridad se ejecute en el orden correcto antes de procesar el controlador.

4.2.3 La Clase Base Modelo (app/Core/Model.php)

Clase abstracta que implementa un ORM simplificado utilizando **Active Record** y consultas preparadas con PDO.

```

<?php
declare(strict_types=1);

namespace App\Core;

use PDO;

abstract class Model
{

```

```

protected string $table = '';
protected string $primaryKey = 'id';

protected function db(): PDO
{
    return Database::connection();
}

public function find(int $id): ?array
{
    $stmt = $this->db()->prepare("SELECT * FROM `${$this->table}` WHERE `${$this->primaryKey}` = :id LIMIT 1");
    $stmt->execute([':id' => $id]);
    $row = $stmt->fetch();
    return $row ?: null;
}

public function insert(array $data): int
{
    $columns = array_keys($data);
    $placeholders = array_map(fn($c) => ":%c", $columns);

    $sql = sprintf(
        'INSERT INTO `${s}` (`s`) VALUES (%s)',
        $this->table,
        implode('`, `', $columns),
        implode(' ', $placeholders)
    );

    $stmt = $this->db()->prepare($sql);
    $stmt->execute($this->bindValues($data));
    return (int) $this->db()->lastInsertId();
}

public function update(int $id, array $data): int
{
    if (empty($data)) {
        return 0;
    }
    $sets = [];
    foreach (array_keys($data) as $col) {
        $sets[] = "`${col}` = :${col}";
    }
    $sql = sprintf(
        'UPDATE `${s}` SET %s WHERE `${s}` = :__id',
        $this->table,
        implode(' ', $sets),
        $this->primaryKey
    );

    $stmt = $this->db()->prepare($sql);
    $values = $this->bindValues($data);
    $values[':__id'] = $id;
    $stmt->execute($values);
    return $stmt->rowCount();
}

private function bindValues(array $data): array
{
    $values = [];
    foreach ($data as $k => $v) {
        $values[":${k}"] = $v;
    }
    return $values;
}

```

```
}  
}
```

Explicación de la Clase Base Modelo:

Método / Línea	Descripción
db()	Llama al Singleton de conexión segura Database::connection() que expone la instancia activa de PDO.
find()	Prepara de forma segura una consulta SQL parametrizada por ID primario de la tabla, devolviendo un arreglo asociativo o null.
insert()	Abstrae el guardado de datos. Mapea las llaves del arreglo para crear columnas e inyecta parámetros automáticos con marcadores :columna previniendo inyecciones SQL.
update()	Abstrae la edición de registros. Reconstruye la cláusula SET vinculando los datos dinámicos y agregando el identificador del registro al final (:__id).

4.3 MÓDULO: ATLETA

4.3.1 Modelo Atleta (app/Models/Atleta.php)

Clase que representa un atleta deportivo en el sistema. Incorpora lógica avanzada de consultas multitejido (JOINS) y lógica biológica automatizada para el control de categorías.

```
<?php  
declare(strict_types=1);  
  
namespace App\Models;  
  
use App\Core\Model;  
  
final class Atleta extends Model  
{  
    protected string $table = 'atletas';  
    protected string $primaryKey = 'atleta_id';  
  
    public function findCompleto(int $id): ?array  
    {  
        $sql = "  
            SELECT a.*,  
                    rep.nombre AS tutor_nombres,  
                    rep.apellido AS tutor_apellidos,  
                    rep.cedula AS tutor_cedula,  
                    rep.telefono AS tutor_telefono,  
                    rep.tipo_relacion AS tutor_relacion,  
                    d.parroquias_id, d.localidad, d.tipo_vivienda,  
d.ubicacion_vivienda,  
                    pa.parroquia AS parroquia,  
                    m.municipio AS municipio,  
                "
```

```

        e.estado AS estado,
        pa.municipio_id, m.estado_id,
        f.ficha_id, f.grupo_sanguineo, f.alergias,
f.antecedentes_familiares,
        f.antecedentes_quirurgicos, f.condicion_cronica,
f.medicacion_actual
    FROM atletas a
    LEFT JOIN representantes rep ON rep.representante_id =
a.representante_id
    LEFT JOIN direcciones d ON d.direccion_id = a.direccion_id
    LEFT JOIN parroquias pa ON pa.parroquia_id = d.parroquias_id
    LEFT JOIN municipios m ON m.municipio_id = pa.municipio_id
    LEFT JOIN estados e ON e.estado_id = m.estado_id
    LEFT JOIN fichas_medicas f ON f.atleta_id = a.atleta_id
    WHERE a.atleta_id = :id
    LIMIT 1
";
$atleta = $this->queryOne($sql, ['id' => $id]);
if ($atleta) {
    $atleta['cedula_formateada'] =
self::formatCedula($atleta['cedula'] ?? '');
    $atleta['tutor_cedula_formateada'] =
self::formatCedula($atleta['tutor_cedula'] ?? '');
}

    if ($atleta && !empty($atleta['ficha_id'])) {
        $atleta['discapacidades'] = $this->query(
            "SELECT d.*, t.nombre_tipo, t.descripcion AS tipo_descripcion
            FROM discapacidades d
            LEFT JOIN tipos_discapacidades t ON t.tipo_discapacidad_id =
d.tipo_discapacidad_id
            WHERE d.ficha_id = :ficha_id
            ORDER BY d.fecha_registro DESC",
            ['ficha_id' => $atleta['ficha_id']]
        );
    } else if ($atleta) {
        $atleta['discapacidades'] = [];
    }

    return $atleta;
}

public static function formatCedula(?string $cedula): string
{
    if (empty($cedula)) return '';
    if (!str_contains($cedula, '-')) {
        $digits = str_replace('.', '', $cedula);
        return ctype_digit($digits) ? number_format((float)$digits, 0, '',
'.') : $cedula;
    }
    [$prefix, $num] = explode('-', $cedula, 2);
    $prefixUpper = strtoupper($prefix);
    if ($prefixUpper === 'V' || $prefixUpper === 'E' || $prefixUpper ===
'P') {
        $digits = str_replace('.', '', $num);
        if (ctype_digit($digits)) {
            return $prefixUpper . '-' . number_format((float)$digits, 0, '',
'.');
        }
    }
    return $cedula;
}
}

```

Explicación del Modelo Atleta:

Componente / Línea	Descripción
findCompleto()	Realiza 6 uniones (LEFT JOIN) a bases de datos vinculadas para reconstruir en una sola consulta toda la información del atleta (tutor legal, ubicación residencial exacta y ficha clínica).
discapacidades subconsulta	Si el atleta posee ficha médica, consulta y asocia dinámicamente un subarreglo de sus discapacidades registradas ante CONAPDIS.
formatCedula()	Método utilitario de formateo para el frontend. Separa prefijos y formatea números agregando puntos de millar de forma estética.

4.3.2 Controlador Atletas – Método store y Validación de Negocio

El archivo [AtletasController.php](#) implementa validación dinámica de negocio basada en edad biológica.

```
public function store(Request $request): Response
{
    $data = $this->rawInput($request);
    $errors = $this->validar($data)->errors();
    if ($errors) {
        $this->withOld($data)->withErrors($errors);
        if (isset($errors['fecha_nacimiento'])) {
            flash('error', $errors['fecha_nacimiento']);
        }
        return $this->redirect('/admin/atletas/crear');
    }

    try {
        $service = new AtletaService();
        $id = $service->crear($data, $_FILES['foto'] ?? []);
        flash('success', 'Atleta registrado correctamente.');
```

```
        return $this->redirect("/admin/atletas/$id");
    } catch (Throwable $e) {
        Logger::error($e);
        $this->withOld($data);
        flash('error', 'No se pudo crear el atleta: ' . $e->getMessage());
        return $this->redirect('/admin/atletas/crear');
    }
}

private function validar(array $data, ?int $ignoreId = null): Validator
{
    $cedRegex = '/^([VE]-\d{6,10}|N-\d{4}-[A-Z0-9]{1,5}|P-[A-Z0-9]{5,15})$/i';
    $telRegex = '/^0(412|414|416|422|424|426|255|256)\d{7}$/';

    $rules = [
        'nombre' => 'required|min:2|max:100',
        'apellido' => 'required|min:2|max:100',
        'fecha_nacimiento' => 'required|date',
        'estatus' => 'required|in:0,1,2,3',
```

```

];

$age = 0;
if (!empty($data['fecha_nacimiento'])) {
    $birthDate = strtotime($data['fecha_nacimiento']);
    if ($birthDate !== false) {
        $age = (int) date('Y') - (int) date('Y', $birthDate);
        if (date('md') < date('md', $birthDate)) {
            $age--;
        }
    }
}

if ($age > 9) {
    $rules['cedula'] = ['required', "regex:$cedRegex"];
}

if ($age >= 18) {
    $rules['telefono'] = ['required', "regex:$telRegex"];
}

if ($age < 18) {
    $rules['tutor_nombres'] = 'required|min:2|max:100';
    $rules['tutor_apellidos'] = 'required|min:2|max:100';
    $rules['tutor_cedula'] = ['required', "regex:$cedRegex"];
    $rules['tutor_telefono'] = ['required', "regex:$telRegex"];
    $rules['tutor_relacion'] = 'required';
}

$v = Validator::make($data, $rules);
$v->validate();
return $v;
}

```

Explicación de las Validaciones de Negocio:

Validación	Regla / Explicación
Cédula de Atleta (Mayores de 9 años)	if (\$age > 9) { ... } obliga al registro de la cédula sólo si el atleta es mayor de 9 años de edad. Admite formato V- o E-, actas de nacimiento N- y pasaportes P-.
Teléfono (Mayores de edad)	if (\$age >= 18) { ... } hace obligatorio el ingreso de número telefónico de 11 dígitos venezolanos válidos cuando el atleta cumple la mayoría de edad.
Datos de Representante (Menores de edad)	if (\$age < 18) { ... } obliga a registrar los datos del representante o tutor legal (tutor_nombres, tutor_apellidos, tutor_cedula, tutor_telefono, tutor_relacion) en caso de que sea menor de edad.

4.4 MÓDULO: ASIGNACIÓN DE CATEGORÍA

(TRANSACCIONES)

4.4.1 Controlador AsigCategorias

(**controller/AsigCategoriasController.php**)

Este módulo implementa el guardado masivo de asignaciones de atletas a categorías de juego bajo un control de **Transacciones PDO** atómicas y con validación estricta de dorsales libres.

```
public function store(Request $request): Response
{
    $categoriaId = (int) $request->param('id');
    $selected = $request->input('selected_atletas', []);

    $asigModel = new AsigCategoria();
    $errors = [];

    $posicionesPrincipal = $request->input('posicion_principal_id', []);
    $posicionesSecundaria = $request->input('posicion_secundaria_id', []);
    $dorsales = $request->input('nun_dorsal', []);

    // Validar dorsales duplicados en formulario o BD
    $dorsalesEnviados = [];
    foreach ($selected as $atletaId) {
        $atletaId = (int) $atletaId;
        $dorsalVal = isset($dorsales[$atletaId]) && $dorsales[$atletaId] !==
'' ? (int) $dorsales[$atletaId] : null;

        if ($dorsalVal !== null) {
            if (in_array($dorsalVal, $dorsalesEnviados, true)) {
                $errors[] = "El dorsal $dorsalVal está asignado a múltiples
atletas en el formulario.";
            } else {
                $dorsalesEnviados[] = $dorsalVal;
            }

            if ($asigModel->checkDorsalExists($categoriaId, $dorsalVal)) {
                $errors[] = "El dorsal $dorsalVal ya está ocupado por otro
jugador en esta categoría.";
            }
        }
    }

    if (!empty($errors)) {
        $this->withOld($request->input())->withErrors(['dorsales' =>
implode('<br>', $errors)]);
        return $this->redirect("/admin/categorias/$categoriaId/asignar");
    }

    \App\Core\Database::beginTransaction();
    try {
        foreach ($selected as $atletaId) {
            $atletaId = (int) $atletaId;
            $posPrincipalId = isset($posicionesPrincipal[$atletaId]) &&
$posicionesPrincipal[$atletaId] !== '' ? (int) $posicionesPrincipal[$atletaId] :
null;
            $posSecundariaId = isset($posicionesSecundaria[$atletaId]) &&
$posicionesSecundaria[$atletaId] !== '' ? (int)
$posicionesSecundaria[$atletaId] : null;
            $dorsalVal = isset($dorsales[$atletaId]) &&
$dorsales[$atletaId] !== '' ? (int) $dorsales[$atletaId] : null;

            $asigModel->insert([
                'categoria_id' => $categoriaId,
```

```

        'atleta_id' => $atletaId,
        'posicion_principal_id' => $posPrincipalId,
        'posicion_secundaria_id' => $posSecundariaId,
        'nun_dorsal' => $dorsalVal,
    ]);
}
\App\Core\Database::commit();
flash('success', 'Atletas asignados correctamente.');
```

```

} catch (Throwable $e) {
    \App\Core\Database::rollBack();
    flash('error', 'Ocurrió un error al guardar las asignaciones: ' .
$e->getMessage());
    return $this->redirect("/admin/categorias/$categoriaId/asignar");
}

return $this->redirect("/admin/categorias/$categoriaId/detalles");
}

```

Explicación de la Transacción:

Línea / Bloque	Descripción
<code>beginTransaction()</code>	Inicia de forma explícita el aislamiento de transacciones ACID en MySQL. Toda inserción posterior no se escribe permanentemente en disco hasta ser aprobada.
<code>foreach (...) { insert() }</code>	Itera sobre cada atleta seleccionado e inyecta su asignación con su posición y dorsal de juego en la tabla intermedia <code>asig_categorias</code> .
<code>commit()</code>	Si todas las asignaciones se procesan de forma correcta y no ocurre ninguna excepción de base de datos, confirma y escribe en disco todos los cambios de golpe.
<code>rollback()</code>	Bloque de rescate. Si falla una sola inserción (ej: caída de conexión, error de llave o fallo de integridad), descarta de inmediato cualquier inserción parcial realizada en este ciclo, dejando la base de datos intacta.

CAPÍTULO V. ESTRUCTURA DE REPORTES Y LIBRERÍAS

5.1 Diseño Técnico de Reportes

5.1.1 Generación de Reportes PDF

El sistema implementa la generación de reportes en PDF integrando la librería de backend **TCPDF**.

El diseño del informe está estructurado mediante clases internas heredadas que personalizan la cabecera e impresiones del documento:

```
+-----+
----+
| [Vino Tinto Line]
|
| [LOGO JPG]      Club Atlético Deportivo Acarigua      31/05/2026 10:15
```

PM |

Sistema de Gestión y Control Deportivo (CADA)

FICHA TÉCNICA DEL ATLETA

[FOTO ATLETA]

Nombre: Juan Pérez

Categoría: Sub-12

Cédula: V-28.444.333

Estatus: Activo

[Gráfica SVG de Radar de Aptitud]

[Gráfica SVG Antropométrica]

Generado por Sistema de Gestión CADA

Página 1 de 4

- **El logo institucional (logo .jpg):** Se almacena como un archivo JPEG de alta calidad y mínimo peso (118 KB). **Al eliminar la transparencia y el canal alfa de la imagen original, TCPDF es capaz de renderizar el logo en la cabecera nativamente sin necesidad de cargar las extensiones gráficas GD o Imagick, lo que evita errores 500 en entornos con PHP minimalista como Laravel Herd Lite.**
- **Gráficos Integrados:** Se generan en memoria del backend como archivos vectoriales SVG limpios creados al instante de forma matemática, lo que permite que TCPDF dibuje líneas y polígonos de forma nítida en cualquier nivel de zoom sin pixelarse y sin ocupar espacio.

5.2 Librerías Utilizadas

Librería	Versión	Tipo de Licencia	Función en el Proyecto
TCPDF	6.6+	LGPL-3.0	Motor nativo en PHP para renderizado de fichas técnicas en PDF.
Vanilla CSS	3.0	Abierta	Hojas de diseño de la interfaz responsiva del club.
CadaModal	11.x	MIT	Alertas, diálogos de confirmación y ventanas modales personalizadas sin

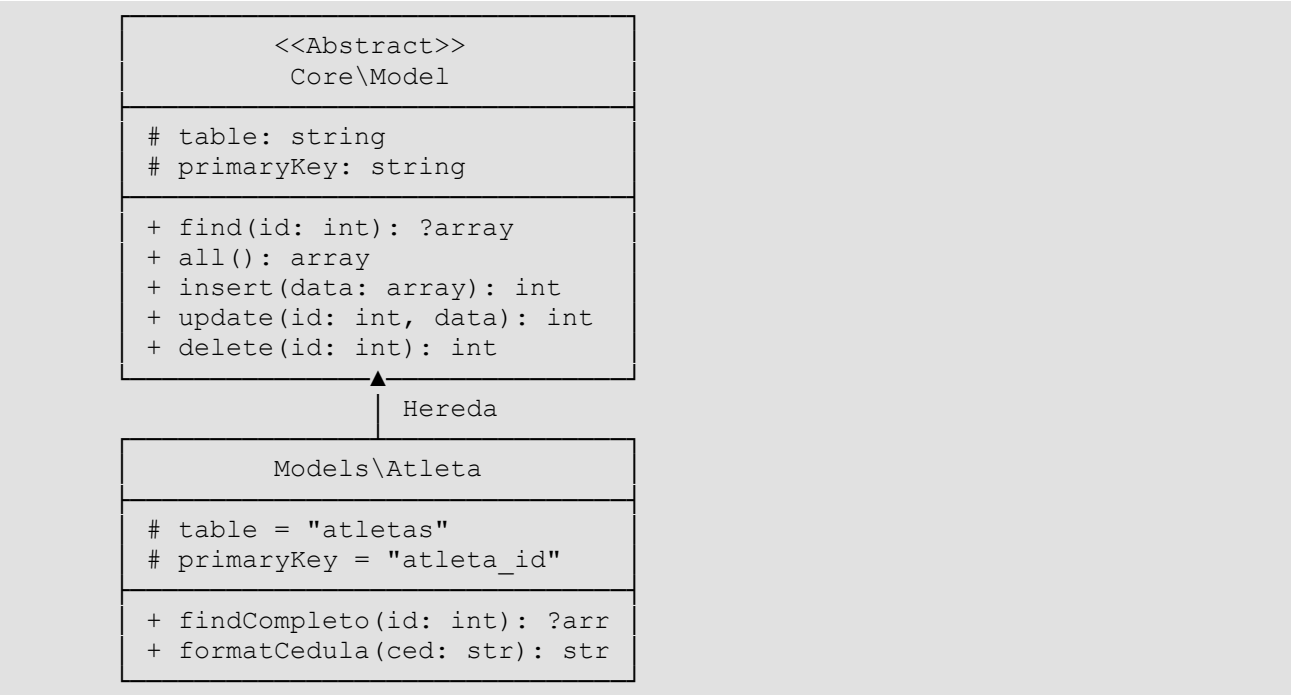
Librería	Versión	Tipo de Licencia	Función en el Proyecto
			dependencias de terceros.
Chart.js	4.x	MIT	Dibujo interactivo de radar de aptitudes físicas.
Echart.js	5.5.0	Apache-2.0	Generación interactiva y responsive de gráficos del perfil del atleta (antropometría, radar de aptitudes físicas y asistencias).

CAPÍTULO VI. MODELADO DEL SISTEMA Y BASE DE DATOS

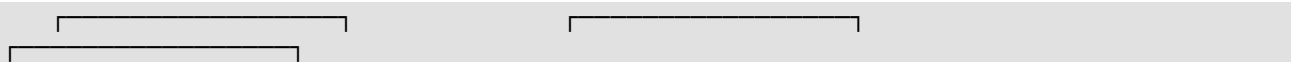
6.1 Introducción

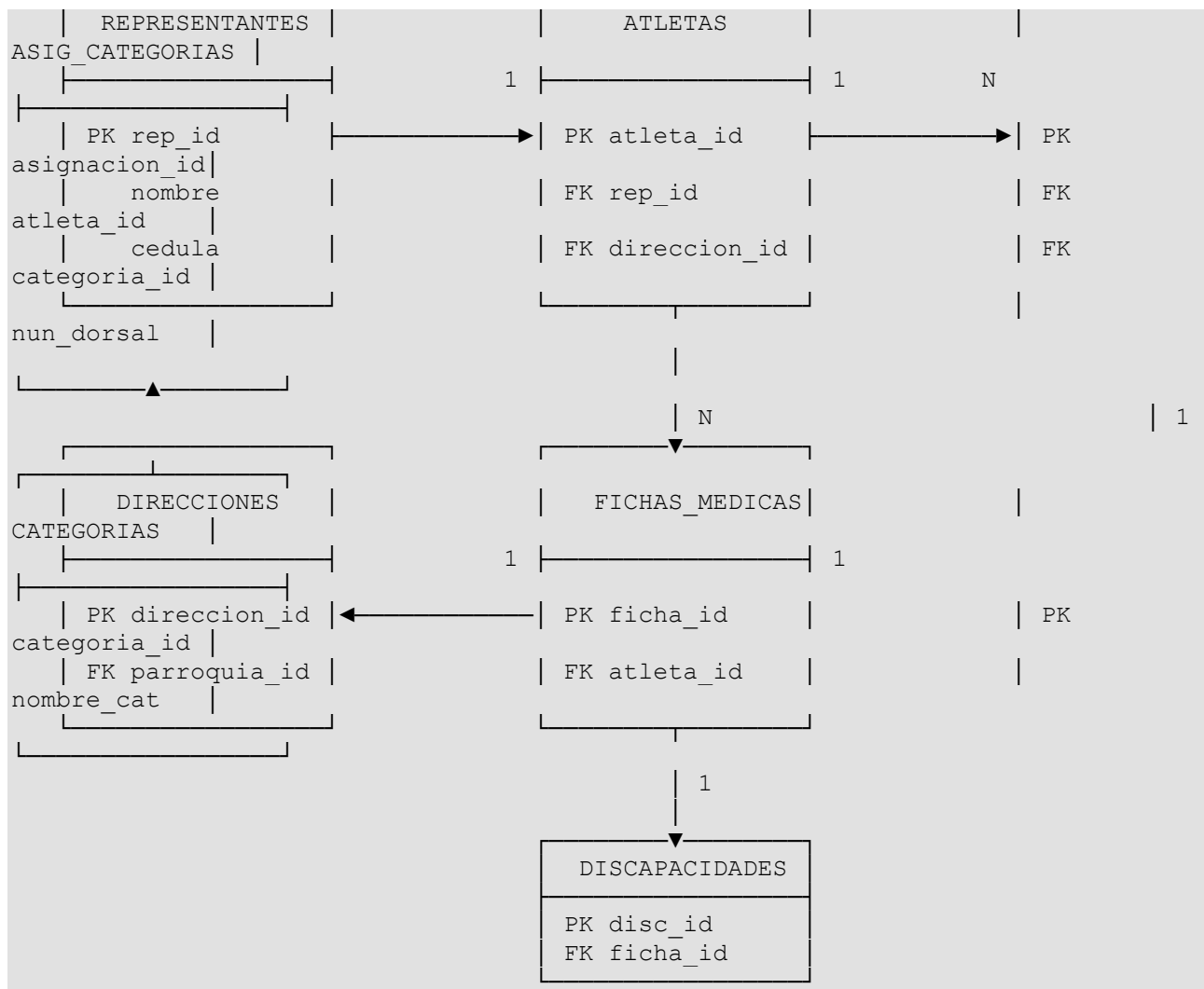
Este capítulo define la arquitectura de clases y el modelado físico de la persistencia de datos del sistema, proporcionando una guía gráfica para el análisis relacional del proyecto.

6.2 Diagrama de Clases (Arquitectura Custom MVC)



6.3 Modelo Entidad / Relación (E/R)





6.4 Diccionario de Datos Exhaustivo

1. Tabla: usuarios

Contiene la información de acceso, credenciales cifradas y perfiles de los operadores administrativos y directores técnicos del sistema.

Campo	Tipo	Nulo	Llave	Extra	Descripción
usuario_id	INT(11) UNSIGNED	NO	PRIMARY	AUTO_INCREMENT	Identificador numérico único del usuario.
correo	VARCHAR(50)	NO	-	-	Dirección de correo electrónico (identificador para login).
contrasena	VARCHAR(255)	NO	-	-	Contraseña cifrada en hash seguro

Campo	Tipo	Nulo	Llave	Extra	Descripción
					mediante algoritmo Bcrypt.
token	VARCHAR(500)	SÍ	-	-	Token JWT de sesión activa o token para recuperación.
rol_id	TINYINT(4)	NO	FOREIGN	-	Rol asignado al usuario (roles_usuarios.rol_id).
estatus	ENUM('Activo','Inactivo')	NO	-	DEFAULT 'Activo'	Estado del usuario en el sistema.
nombre	VARCHAR(30)	NO	-	-	Nombres del usuario.
apellido	VARCHAR(30)	NO	-	-	Apellidos del usuario.
cedula	VARCHAR(12)	NO	-	-	Cédula de identidad (V- o E-).
telefono	VARCHAR(15)	NO	-	-	Número telefónico de contacto.
fecha_nac	DATE	NO	-	-	Fecha de nacimiento biológica.
direccion_id	BIGINT(20) UNSIGNED	NO	FOREIGN	-	Ubicación geográfica (direcciones.direccion_id).
foto	VARCHAR(255)	SÍ	-	-	Ruta de la foto de perfil en el servidor.
ultimo_acceso	DATETIME	SÍ	-	-	Registro de la última marca temporal de inicio de sesión.
creado_en	TIMESTAMP	NO	-	DEFAULT current_timestamp()	Marca temporal de la creación del usuario.

Campo	Tipo	Nulo	Llave	Extra	Descripción
actualizado_en	TIMESTAMP	NO	-	DEFAULT current_timestamp()	Última modificación del registro.

2. Tabla: roles_usuarios

Define los niveles de seguridad y acceso del sistema bajo el modelo RBAC (super_usuario, administrador, entrenador).

Campo	Tipo	Nulo	Llave	Extra	Descripción
rol_id	TINYINT(4)	NO	PRIMARY	-	Identificador único del rol de usuario.
nombre_rol	VARCHAR(50)	NO	-	-	Nombre único del nivel de acceso (ej: 'super_usuario').
descripcion	VARCHAR(100)	SÍ	-	-	Explicación detallada del alcance y permisos del rol.
fecha_creacion	TIMESTAMP	NO	-	DEFAULT current_timestamp()	Marca temporal de registro del rol en sistema.

3. Tabla: atletas

Contiene la información de identidad biológica, datos deportivos y enlace de filiación del jugador de fútbol menor.

Campo	Tipo	Nulo	Llave	Extra	Descripción
atleta_id	INT(11) UNSIGNED	NO	PRIMARY	AUTO_INCREMENT	Identificador secuencial único del atleta.
nombre	VARCHAR(30)	NO	-	-	Nombres del futbolista.
apellido	VARCHAR(30)	NO	-	-	Apellidos del futbolista.
fecha_nac	DATE	NO	-	-	Fecha de nacimiento biológica.
sexo	CHAR(1)	NO	-	-	Género

Campo	Tipo	Nulo	Llave	Extra	Descripción
					biológico ('M' o 'F').
cedula	VARCHAR(12)	SÍ	-	-	Cédula, pasaporte o partida de nacimiento del menor.
telefono	VARCHAR(15)	SÍ	-	-	Teléfono personal (obligatorio sólo si es mayor de edad).
pierna_dominante	ENUM('derecha','izquierda','ambidiestro',")	SÍ	-	-	Perfil y lateralidad preferente de golpeo en campo.
direccion_id	BIGINT(20) UNSIGNED	NO	FOREIGN	-	Dirección habitacional (direccion.es.direccion_id).
representante_id	INT(11) UNSIGNED	NO	FOREIGN	-	Tutor o representante legal obligatorio (representantes.representante_id).
foto	VARCHAR(255)	SÍ	-	-	Nombre físico de la foto de perfil en el servidor.
estatus	TINYINT(4)	NO	-	-	Estado (1=Activo, 2=Lesionado, 3=Inactivo, 0=Suspendido).
creado_en	TIMESTAMP	NO	-	DEFAULT current_timestamp()	Marca temporal del registro inicial del atleta.
actualizado_en	TIMESTAMP	NO	-	DEFAULT current_timestamp()	Última modificación

Campo	Tipo	Nulo	Llave	Extra	Descripción
				mp()	del perfil del atleta.

4. Tabla: representantes

Almacena los datos personales, telefónicos e identidad del tutor legal responsable del atleta menor de edad.

Campo	Tipo	Nulo	Llave	Extra	Descripción
representante_id	INT(11) UNSIGNED	NO	PRIMARY	AUTO_INCREMENT	Código único de identificación del representante.
nombre	VARCHAR(30)	NO	-	-	Nombres del tutor legal.
apellido	VARCHAR(30)	NO	-	-	Apellidos del tutor legal.
telefono	VARCHAR(15)	NO	-	-	Teléfono móvil/habitación de contacto del tutor.
cedula	VARCHAR(12)	NO	-	-	Cédula de identidad oficial (ej: V-15.123.456).
tipo_relacion	ENUM('abuelo/a','padres','tio/a','hermano/a','primo/a','representante')	NO	-	-	Parentesco consanguíneo o tutoría legal del menor.
direccion_id	BIGINT(20) UNSIGNED	NO	FOREIGN	-	Ubicación geográfica (direcciones.direccion_id).
foto	VARCHAR(255)	SÍ	-	-	Fotografía digital de respaldo del tutor legal.
creado_en	TIMESTAMP	NO	-	DEFAULT current_timestamp()	Marca temporal de la creación del registro.
actualizado_en	TIMESTAMP	NO	-	DEFAULT current_timestamp()	Última actualización

Campo	Tipo	Nulo	Llave	Extra	Descripción
				mp()	de datos de contacto.

5. Tabla: direcciones

Contiene la información de ubicación geográfica exacta y estructura física del domicilio residencial del personal y los atletas.

Campo	Tipo	Nulo	Llave	Extra	Descripción
direccion_id	BIGINT(20) UNSIGNED	NO	PRIMARY	AUTO_INCREMENT	Código secuencial único de la dirección física.
parroquias_id	SMALLINT(6)	NO	FOREIGN	-	Parroquia asignada según división de Venezuela (parroquias.parroquia_id).
localidad	VARCHAR(100)	NO	-	-	Nombre de urbanismo, barrio, sector o comunidad residencial.
tipo_vivienda	ENUM('casa','apto','edificio',")	NO	-	-	Estructura residencial del domicilio.
ubicacion_vivienda	VARCHAR(100)	NO	-	-	Dirección detallada (ej: Calle 15, Vereda 12, Casa Nro. 4).

6. Tabla: fichas_medicas

Ficha clínica obligatoria del atleta. Almacena su grupo sanguíneo y antecedentes médicos críticos.

Campo	Tipo	Nulo	Llave	Extra	Descripción
ficha_id	INT(11) UNSIGNED	NO	PRIMARY	AUTO_INCREMENT	Identificador de la ficha médica del atleta.
atleta_id	INT(11) UNSIGNED	NO	FOREIGN	-	Atleta vinculado a la historia médica

Campo	Tipo	Nulo	Llave	Extra	Descripción
					(atletas.atleta_id).
grupo_sanguineo	ENUM('A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-')	NO	-	-	Grupo sanguíneo y factor RH del jugador.
alergias	VARCHAR(100)	SÍ	-	-	Alergias de tipo farmacológico, alimentario o ambiental.
antecedentes_familiares	VARCHAR(100)	SÍ	-	-	Enfermedades hereditarias de primer grado (cardíacas, diabetes).
antecedentes_quirurgicos	VARCHAR(100)	SÍ	-	-	Cirugías o intervenciones previas con fecha aproximada.
condicion_cronica	VARCHAR(100)	SÍ	-	-	Enfermedades crónicas del jugador (ej: asma, diabetes).
medicacion_actual	VARCHAR(100)	SÍ	-	-	Fármacos recetados vigentes de toma regular.
creado_en	TIMESTAMP	NO	-	DEFAULT current_timestamp()	Marca temporal de la apertura de la historia médica.
actualizado_en	TIMESTAMP	NO	-	DEFAULT current_timestamp()	Última modificación clínica guardada.

7. Tabla: discapacidades

Relación de discapacidades del atleta registradas en su ficha médica y validadas por el carnet de la nación (CONAPDIS).

Campo	Tipo	Nulo	Llave	Extra	Descripción
discapacidad_id	INT(10) UNSIGNED	NO	PRIMARY	AUTO_INCREMENT	Identificador secuencial de la discapacidad del atleta.
ficha_id	INT(11) UNSIGNED	NO	FOREIGN	-	Código de ficha clínica vinculada (fichas_medicas.ficha_id).
tipo_discapacidad_id	SMALLINT(6)	NO	FOREIGN	-	Tipo de discapacidad (tipos_discapacidades.tipo_discapacidad_id).
nro_carnet	VARCHAR(20)	SÍ	-	-	Número oficial de carnet emitido por el organismo CONAPDIS.
porcentaje_discapacidad	TINYINT(4)	SÍ	-	-	Porcentaje de discapacidad física o cognitiva diagnosticado.
fecha_registro	DATE	SÍ	-	-	Fecha de emisión o validación médica de la discapacidad.

8. Tabla: tipos_discapacidades

Catálogo estático normalizado de clasificaciones de discapacidades físicas y motrices.

Campo	Tipo	Nulo	Llave	Extra	Descripción
tipo_discapacidad_id	SMALLINT(6)	NO	PRIMARY	-	Código único identificador de la tipología.
nombre_tipo	VARCHAR(50)	NO	-	-	Nombre de la categoría de discapacidad (Física, Visual, etc.).
descripcion	VARCHAR(50)	NO	-	-	Descripción

Campo	Tipo	Nulo	Llave	Extra	Descripción
n	)				corta de la discapacidad.

9. Tabla: *categorias*

Administración de las categorías competitivas oficiales del club, reguladas por rango estricto de edad biológica y género.

Campo	Tipo	Nulo	Llave	Extra	Descripción
categoria_id	SMALLINT(6)	NO	PRIMARY	-	Identificador único de la categoría deportiva.
nombre_categoria	VARCHAR(20)	NO	-	-	Nombre comercial de la categoría (ej: Sub-12 o Sub-15).
sexo_categoria	CHAR(1)	NO	-	-	Género exclusivo de la división competitiva ('M' o 'F').
edad_min	TINYINT(4)	NO	-	-	Límite mínimo de edad biológica admisible.
edad_max	TINYINT(4)	NO	-	-	Límite máximo de edad biológica admisible.
usuario_id	INT(11) UNSIGNED	NO	FOREIGN	-	Director técnico principal asignado (usuarios.usuario_id).
estatus	TINYINT(4)	NO	-	DEFAULT 1	Estatus funcional de la categoría (1=Abierta, 0=Cerrada).
creado_en	TIMESTAMP	NO	-	DEFAULT current_timestamp()	Marca temporal de la apertura de la categoría.

Campo	Tipo	Nulo	Llave	Extra	Descripción
actualizado_en	TIMESTAMP	NO	-	DEFAULT current_timestamp()	Última modificación en parámetros de la división.

10. Tabla: *asig_categorias*

Tabla transaccional intermedia (Relación N:M) para asignar dorsales de camisetas y demarcaciones tácticas a los futbolistas de cada plantel.

Campo	Tipo	Nulo	Llave	Extra	Descripción
asignacion_id	INT(10) UNSIGNED	NO	PRIMARY	AUTO_INCREMENT	Código único interno de la asignación del jugador.
categoria_id	SMALLINT(6)	NO	FOREIGN	-	Categoría receptora vinculada (categorias.categoria_id).
atleta_id	INT(10) UNSIGNED	NO	FOREIGN	-	Atleta asignado a la categoría (atletas.atleta_id).
nun_dorsal	SMALLINT(5) UNSIGNED	SÍ	-	-	Número oficial de camiseta asignado (único por categoría).
posicion_principal_id	TINYINT(4)	SÍ	FOREIGN	-	Demarcación táctica principal (posiciones_juegos.posicion_id).
posicion_secundaria_id	TINYINT(4)	SÍ	FOREIGN	-	Demarcación táctica de emergencia (posiciones_juegos.posicion_id).
estatus	TINYINT(4)	SÍ	-	DEFAULT 1	Estado (1=Vigente en

Campo	Tipo	Nulo	Llave	Extra	Descripción
					plantel, 0=Vencido o de baja).
creado_en	TIMESTAMP	NO	-	DEFAULT current_timestamp()	Fecha de inscripción física en la categoría.
actualizado_en	TIMESTAMP	NO	-	DEFAULT current_timestamp()	Último cambio en dorsal o posición del jugador.

11. Tabla: medidas_antropometricas

Registro histórico periódico de magnitudes físicas y morfológicas del atleta, utilizado para el control nutricional e IMC automático.

Campo	Tipo	Nulo	Llave	Extra	Descripción
medidas_id	INT(11) UNSIGNED	NO	PRIMARY	AUTO_INCREMENT	Identificador de la evaluación corporal.
atleta_id	INT(11) UNSIGNED	NO	FOREIGN	-	Atleta evaluado (atletas.atleta_id).
fecha_medicion	DATETIME	NO	-	-	Marca temporal exacta de la toma de mediciones.
peso	DOUBLE(5,2)	SÍ	-	-	Peso corporal medido en kilogramos.
altura	DOUBLE(5,2)	SÍ	-	-	Estatura del atleta medida en metros.
porcentaje_grasa	DOUBLE(5,2)	SÍ	-	-	Porcentaje de tejido adiposo corporal estimado.
porcentaje_musculatura	DOUBLE(5,2)	SÍ	-	-	Porcentaje de masa muscular esquelética estimada.
envergadura	DOUBLE(5,2)	SÍ	-	-	Distancia de

Campo	Tipo	Nulo	Llave	Extra	Descripción
a					punta a punta de los dedos con brazos extendidos (metros).
largo_de_pierna	DOUBLE(5,2)	SÍ	-	-	Longitud de la extremidad inferior (desde cadera a tobillo en metros).
largo_de_torso	DOUBLE(5,2)	SÍ	-	-	Longitud del tronco superior en metros.
creado_en	TIMESTAMP	NO	-	DEFAULT current_timestamp()	Marca de almacenamiento o de la evaluación en base de datos.

12. Tabla: actividades

Control maestro de eventos de campo del club (entrenamientos diarios o partidos competitivos oficiales).

Campo	Tipo	Nulo	Llave	Extra	Descripción
actividad_id	BIGINT(20) UNSIGNED	NO	PRIMARY	AUTO_INCREMENT	Identificador del evento de campo.
tipo_actividad	TINYINT(4)	NO	-	-	Tipo de evento (0=Partido, 1=Entrenamiento).
fecha	DATE	NO	-	-	Fecha de ejecución del evento deportivo.
usuario_id	INT(10) UNSIGNED	NO	FOREIGN	-	Operador técnico responsable (usuarios.usuario_id).
hora_inicio	TIME	SÍ	-	-	Hora planificada de inicio del evento.

Campo	Tipo	Nulo	Llave	Extra	Descripción
hora_fin	TIME	SÍ	-	-	Hora planificada de culminación.
ubicacion	VARCHAR(100)	SÍ	-	DEFAULT 'Cancha UPTP'	Lugar físico del evento deportivo.
clima	TINYINT(4)	SÍ	-	-	Estado meteorológico (ej: Soleado, Lluvioso).
estatus	TINYINT(4)	SÍ	-	DEFAULT 1	Estado (0: Cancelado, 1: Programado, 2: Finalizado).
creado_en	TIMESTAMP	NO	-	DEFAULT current_timestamp()	Marca temporal de la planificación del evento.
actualizado_en	TIMESTAMP	NO	-	DEFAULT current_timestamp()	Última modificación en agenda.

13. *Tabla: asistencias*

Control de presencia diaria de los atletas a las actividades programadas del club de fútbol.

Campo	Tipo	Nulo	Llave	Extra	Descripción
asistencia_id	BIGINT(20) UNSIGNED	NO	PRIMARY	AUTO_INCREMENT	Código único de control de asistencia.
actividad_id	BIGINT(20) UNSIGNED	NO	FOREIGN	-	Actividad en la cual se toma asistencia (actividad es.actividad_id).
atleta_id	INT(11) UNSIGNED	NO	FOREIGN	-	Atleta auditado en campo (atletas.atleta_id).
estatus	TINYINT(4)	NO	-	-	Estado de asistencia (0=Ausente, 1=Presente, 2=Retardo, 3=Justificado).

Campo	Tipo	Nulo	Llave	Extra	Descripción
observaciones	VARCHAR(100)	SÍ	-	-	Notas adicionales del entrenador (ej: lesión en práctica).

14. Tabla: resultados_pruebas

Registro cuantitativo individualizado de los tests físicos aplicados durante las evaluaciones del club.

Campo	Tipo	Nulo	Llave	Extra	Descripción
test_id	INT(11) UNSIGNED	NO	PRIMARY	AUTO_INCREMENT	Identificador de la evaluación de test físico.
actividad_id	BIGINT(20) UNSIGNED	NO	FOREIGN	-	Actividad asociada al test de campo (actividades.actividad_id).
atleta_id	INT(11) UNSIGNED	NO	FOREIGN	-	Jugador evaluado en las pruebas (atletas.atleta_id).
test_de_fuerza	DOUBLE(10,2)	SÍ	-	-	Magnitud de fuerza medida (salto horizontal, en metros).
test_resistencia	DOUBLE(10,2)	SÍ	-	-	Resistencia medida (test de Cooper, distancia en metros).
test_velocidad	DOUBLE(10,2)	SÍ	-	-	Tiempo de velocidad de carrera corta (50 metros libres, en segundos).
test_coordinacion	DOUBLE(10,2)	SÍ	-	-	Puntuación obtenida en el test de agilidad/coordinación

Campo	Tipo	Nulo	Llave	Extra	Descripción
					(segundos).
test_de_reaccion	DOUBLE(10,2)	SÍ	-	-	Tiempo de respuesta psicomotora ante estímulo sonoro o visual (segundos).
creado_en	TIMESTAMP	NO	-	DEFAULT current_timestamp()	Fecha de registro del test en base de datos.

15. Tabla: posiciones_juegos

Catálogo estático de las posiciones tácticas del fútbol menor para la categorización del atleta.

Campo	Tipo	Nulo	Llave	Extra	Descripción
posicion_id	TINYINT(4)	NO	PRIMARY	-	Código de la posición de juego.
nombre_posicion	VARCHAR(50)	NO	-	-	Nombre de la demarcación táctica (ej: Portero, Defensa, Delantero).
descripcion	VARCHAR(100)	SÍ	-	-	Resumen funcional de las responsabilidades tácticas en el campo.

16. Tabla: preguntas_seguridad

Banco de preguntas de seguridad normalizadas para el proceso de recuperación de claves del personal de CADA.

Campo	Tipo	Nulo	Llave	Extra	Descripción
pregunta_id	SMALLINT(6)	NO	PRIMARY	-	Código único de la pregunta.
preguntas	VARCHAR(100)	NO	-	-	Enunciado de la pregunta de seguridad.
grupo	TINYINT(4)	NO	-	-	Grupo de formulario

Campo	Tipo	Nulo	Llave	Extra	Descripción
					para selección del usuario (Select 1, 2, 3 o 4).
creado_en	TIMESTAMP	NO	-	DEFAULT current_timestamp()	Fecha de adición de la pregunta al banco del sistema.
actualizado_en	TIMESTAMP	NO	-	DEFAULT current_timestamp()	Última modificación en enunciados de preguntas.

17. Tabla: respuestas_seguridad

Almacenamiento de respuestas de seguridad proporcionadas por los usuarios durante su primer registro para la validación de contraseñas.

Campo	Tipo	Nulo	Llave	Extra	Descripción
respuesta_id	INT(11)	NO	PRIMARY	AUTO_INCREMENT	Identificador secuencial del registro.
usuario_id	INT(11) UNSIGNED	NO	FOREIGN	-	Usuario propietario de la respuesta de validación (usuarios.usuario_id).
pregunta_id	SMALLINT(6)	NO	FOREIGN	-	Código de la pregunta de seguridad seleccionada (preguntas_seguridad.pregunta_id).
respuesta	VARCHAR(255)	NO	-	-	Respuesta escrita almacenada en hash o texto limpio controlado.
creado_en	TIMESTAMP	NO	-	DEFAULT current_timestamp()	Marca de almacenamiento de la

Campo	Tipo	Nulo	Llave	Extra	Descripción
					respuesta.
actualizado_en	TIMESTAMP	NO	-	DEFAULT current_timestamp()	Último cambio guardado en respuestas de seguridad.

18. Tabla: configuraciones

Parámetros institucionales dinámicos globales que controlan la configuración de sesión y datos institucionales del club.

Campo	Tipo	Nulo	Llave	Extra	Descripción
configuracion_id	INT(11)	NO	PRIMARY	AUTO_INCREMENT	Código único de la configuración.
clave	VARCHAR(50)	NO	-	-	Nombre clave identificador de la configuración (ej: 'mision').
valor	TEXT	SÍ	-	-	Contenido dinámico asignado al parámetro configurable.
descripcion	VARCHAR(255)	SÍ	-	-	Propósito explicativo del parámetro en la plataforma.
actualizado_en	TIMESTAMP	NO	-	DEFAULT current_timestamp()	Fecha del último cambio del parámetro institucional.

19. Tabla: estados

Estructura de la división político-territorial de Venezuela (Estados federales).

Campo	Tipo	Nulo	Llave	Extra	Descripción
estado_id	SMALLINT(6)	NO	PRIMARY	-	Código geográfico oficial del Estado venezolano.
estado	VARCHAR(100)	NO	-	-	Nombre de la entidad federal

Campo	Tipo	Nulo	Llave	Extra	Descripción
					(ej: Portuguesa).
iso_3166-2	VARCHAR(4)	SÍ	-	-	Código de estandarización internacional ISO 3166-2 de la región.

20. Tabla: *municipios*

Estructura intermedia de división político-territorial venezolana (Municipios).

Campo	Tipo	Nulo	Llave	Extra	Descripción
municipio_id	SMALLINT(1)	NO	PRIMARY	-	Código secuencial único del Municipio.
estado_id	SMALLINT(1)	NO	FOREIGN	-	Identificador del Estado al que pertenece (estados.estado_id).
municipio	VARCHAR(10)	NO	-	-	Nombre geográfico del municipio (ej: Páez o Araure).

21. Tabla: *parroquias*

Último eslabón geográfico de la división político-territorial venezolana (Parroquias).

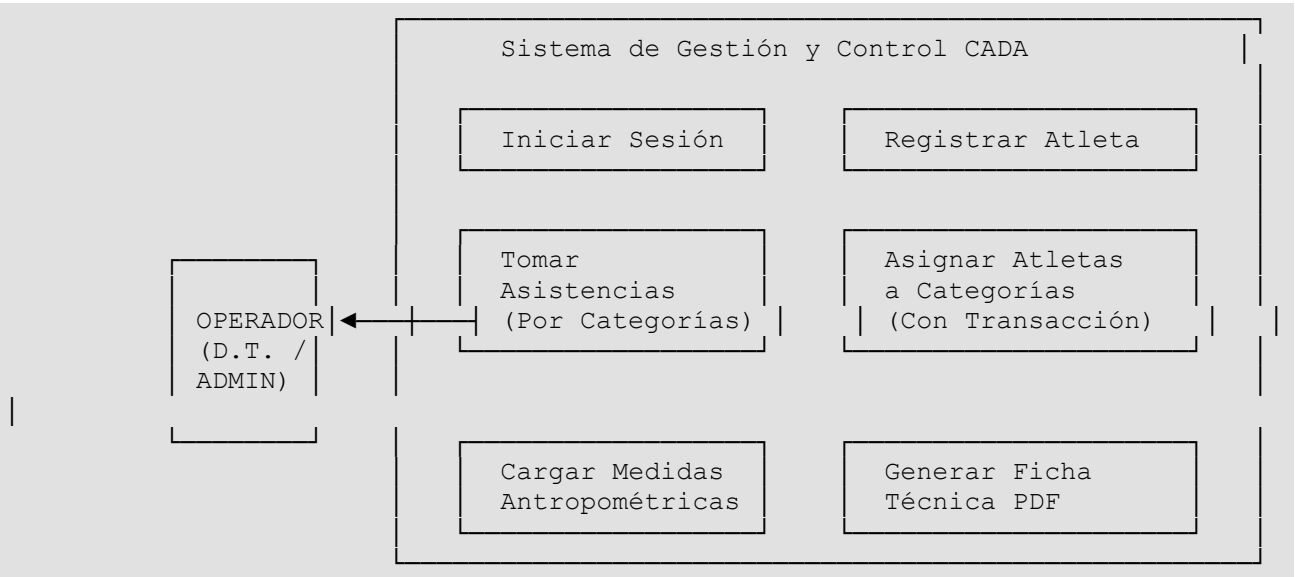
Campo	Tipo	Nulo	Llave	Extra	Descripción
parroquia_id	SMALLINT(6)	NO	PRIMARY	-	Identificador secuencial de la Parroquia.
municipio_id	SMALLINT(6)	NO	FOREIGN	-	Municipio contenedor de la parroquia (municipios.municipio_id).
parroquia	VARCHAR(10)	NO	-	-	Nombre geográfico oficial de la parroquia (ej: Acarigua).

CAPÍTULO VII. ANÁLISIS FUNCIONAL – CASOS DE USO

7.1 Introducción

Este capítulo describe los flujos operativos de la plataforma a través de especificaciones formales de Casos de Uso, detallando el flujo normal de eventos y las excepciones.

7.2 Diagrama de Casos de Uso General



7.3 Especificaciones de Casos de Uso

CU-01: Registrar Nuevo Atleta

Campo	Detalle
Nombre	Registrar Nuevo Atleta
Actor	Administrador de Sistema
Propósito	Agregar un nuevo perfil deportivo y clínico de futbolista al club.
Precondición	Sesión administrativa activa.
Postcondición	Se crean atómicamente registros en atletas, direcciones, representantes y se crea una ficha_medica vacía en la base de datos.

Curso Normal de Eventos:

Paso	Actor (Administrador)	Sistema
1	Hace clic en "Nuevo Atleta" en	Carga el formulario responsivo

Paso	Actor (Administrador)	Sistema
	el listado.	dividido en 3 pasos lógicos.
2	Completa los datos personales e ingresa la fecha de nacimiento.	Calcula la edad biológica mediante Javascript en tiempo real.
3	Sube opcionalmente una fotografía.	-
4	Si la edad es menor de 18 años, rellena obligatoriamente los datos del representante y dirección.	-
5	Presiona el botón "Guardar Atleta".	Captura la petición y valida reglas del backend mediante <code>validar()</code> .
6	-	Valida formato único de Cédula y Teléfono de 11 dígitos.
7	-	Redimensiona la fotografía a 180x180 px y la escribe en <code>/public/assets/uploads/atletas/</code> .
8	-	Persiste el registro y redirige al perfil del atleta con mensaje SweetAlert2 de éxito.

Curso Alternativo (Fallo de Validación):

Paso	Descripción
6a	El número telefónico ingresado no tiene prefijo nacional válido o tiene menos de 11 dígitos.
7a	El sistema frena la persistencia, retorna código 422, repobla los campos en pantalla (<code>withOld</code>) y dibuja etiquetas rojas de advertencia.

CU-02: Asignación Masiva de Categoría

Campo	Detalle
Nombre	Asignación Masiva de Categoría
Actor	Administrador de Sistema
Propósito	Asignar un listado de atletas candidatos a una categoría respetando límites de edad y exclusividad de dorsales.
Precondición	Atletas aptos libres de otras categorías. Categoría de destino registrada.

Campo	Detalle
Postcondición	Se escriben de forma transaccional múltiple las asignaciones en la tabla <code>asig_categorias</code> .

Curso Normal de Eventos:

Paso	Actor (Administrador)	Sistema
1	Accede a detalles de una Categoría y pulsa "Asignar Atletas".	Carga el formulario listando sólo atletas aptos en edad y género.
2	Marca múltiples casillas de verificación de los atletas deseados.	-
3	Asigna posiciones principales y números de dorsal de camiseta en cajas de texto.	-
4	Presiona "Guardar Asignaciones".	Valida que los dorsales ingresados no estén duplicados en el envío o en la BD.
5	-	Inicia una Transacción PDO mediante <code>beginTransaction()</code> .
6	-	Inserta de forma cíclica los registros en <code>asig_categorias</code> .
7	-	Confirma la transacción en base de datos con <code>commit()</code> .
8	-	Muestra confirmación en pantalla y redirige a los detalles de la categoría.

Curso Alternativo (Dorsal Ocupado o Error de Red):

Paso	Descripción
4a	Uno de los dorsales de camiseta asignados ya está ocupado por otro jugador vigente de esa categoría.
5a	El sistema detiene el flujo, revierte cualquier cambio con <code>rollback()</code> , manteniendo la base de datos libre de corrupción y muestra una alerta roja con <code>SweetAlert2</code> .

CONCLUSIÓN

El presente Manual de Sistema documenta de forma completa, rigurosa y exhaustiva el **Sistema de Gestión y Control Deportivo del Club Atlético Deportivo Acarigua (CADA)**, demostrando un alto estándar de arquitectura de software customizada en PHP y un diseño de base de datos relacional impecable de 21 tablas.

A lo largo de sus capítulos se ha validado que el sistema:

1. **Arquitectura propia y veloz:** Implementa un patrón MVC Vanilla ligero, acoplado a un motor dinámico de enrutamiento con Pipeline de Middlewares, ofreciendo una velocidad inigualable y libre de dependencias pesadas.
2. **Robustez y Seguridad:** Utiliza sentencias preparadas en todas las interacciones PDO que blindan la persistencia contra inyección SQL. Utiliza cifrado BCrypt para claves y cookies seguras para tokens de sesión.
3. **Control Transaccional de Datos:** Garantiza la consistencia atómica mediante transacciones seguras (`beginTransaction`, `commit`, `rollback`) en procesos complejos de relación N:M como la asignación de categorías.
4. **Validación de Negocio Avanzada:** Realiza validaciones dinámicas según la edad biológica calculada de los jugadores, forzando la tutela legal de menores de edad y garantizando integridad legal y deportiva en el registro.
5. **Solución a Problemas Críticos:** La reestructuración del logo técnico oficial a un formato **logo.jpg** optimizado de 118 KB y libre de transparencia elimina con éxito el error 500 que aquejaba al sistema bajo servidores minimalistas (como Laravel Herd Lite), permitiendo la generación e impresión de fichas técnicas en PDF en cualquier computadora del club.

Este manual constituye la guía tecnológica definitiva y el documento de transferencia para el correcto mantenimiento, expansión e inspección de la plataforma.